

Guía de Estudio N°3

Conceptos Básicos de Python

Ingeniería Civil en Informática - Departamento de Ciencias de la Ingeniería
Programación - Semestre I

Docente: Víctor Saldivia Vera

28 de Mayo de 2026

Objetivo

Esta Guía de Estudio tiene como propósito consolidar el contenido inicial del Resultado de Aprendizaje 2 (RA2) de la asignatura, permitiendo al estudiante expresar soluciones algorítmicas iniciales bajo el paradigma imperativo mediante un lenguaje de programación.

A través de una metodología secuencial, los ejercicios guiarán al alumno en la correcta declaración de variables, la manipulación de tipos de datos primitivos (*int*, *float*, *complex*) y la aplicación práctica de funciones esenciales de estructuras indexadas y cadenas de texto (*strings*). Con esto se busca fortalecer el razonamiento analítico y la capacidad técnica en la resolución de problemas de ingeniería antes de incorporar estructuras de control de flujo complejas.

Ejercicios Propuestos

1. En el análisis de antenas y redes de telecomunicaciones, la impedancia de una línea de transmisión se compone de una parte real (resistencia) y una parte imaginaria (reactancia). Un ingeniero necesita calcular la impedancia total sumando los componentes de dos tramos de la red de fibra óptica de la universidad.
 - a) Defina la impedancia del Tramo 1 como un número complejo con parte real 50 y parte imaginaria 30 ($50 + 30j$).
 - b) Defina la impedancia del Tramo 2 de la misma forma, con parte real 40 y parte imaginaria -10 ($40 - 10j$).
 - c) Calcule la impedancia total sumando ambos tramos.
 - d) Muestre en pantalla la impedancia total, y luego imprima por separado solo la parte real (convertida a número entero *int*) y la parte imaginaria (convertida a *int*) usando los atributos *.real* y *.imag*.

2. Al desarrollar sistemas informáticos, los usuarios suelen ingresar datos con espacios accidentales o formatos incorrectos. El sistema de la biblioteca de la ULagos recibió el RUT de un estudiante, pero viene “sucio” con espacios al inicio, al final y con puntos intermedios: “ 19.543.872-K ”...

Escribe un programa que:

- a) Guarde el RUT original en una variable de tipo *string*.
 - b) Utilice el método propio de Python para eliminar los espacios en blanco de los extremos.
 - c) Utilice un método propio de Python para eliminar los puntos (.).
 - d) Calcule el largo total del RUT ya limpio (sin espacios ni puntos) y muestre el resultado por pantalla junto al RUT con su nuevo formato.
3. El Departamento de Admisión de la Universidad requiere un script básico para registrar correos institucionales. El programa debe solicitar al usuario que ingrese su nombre completo por terminal. Debido a que los usuarios pueden escribir con mayúsculas y minúsculas desordenadas o con espacios de más, el programa debe estandarizar el texto. Escribe un programa que:

- a) Solicite por terminal el nombre del estudiante.
 - b) Remueva los espacios sobrantes de los extremos.
 - c) Convierta todo el texto a minúsculas.
 - d) Reemplace los espacios intermedios por puntos (.) para simular la estructura de un correo electrónico.
 - e) Muestre en pantalla el resultado final con el texto *@alumnos.ulagos.cl* concatenado al final.
4. En física de partículas, la precisión de los decimales es crítica. Un sensor de presión hidráulica en un laboratorio de la universidad entrega una medida de *1024.7689 Pascals* como tipo *float*. Escribe un programa que realice lo siguiente:
 - a) Defina la variable con el valor del sensor.
 - b) Convierta dicho valor a un número entero (*int*), descartando los decimales, y almacénalo en una variable nueva.
 - c) Utilice un método propio de Python para redondear el valor original del sensor a exactamente 2 decimales y guarde el resultado en otra variable.
 - d) Imprima un mensaje comparativo que muestre por terminal: el valor original, el valor truncado como entero y el valor redondeado.

5. Una plataforma web de la Universidad de Los Lagos mide la velocidad de respuesta de su servidor de asignación de asignaturas. Se han tomado 3 muestras de tiempo de respuesta (en milisegundos) de forma manual. Escribe un programa en Python que:
- a)* Solicite al administrador de la plataforma ingresar por terminal los 3 tiempos de respuesta (los cuales pueden contener decimales, tipo *float*).
 - b)* Almacene los 3 valores ingresados dentro de una lista de Python que debe tener por nombre *tiempos_respuesta*.
 - c)* Acceda por medio de sus índices (*[0]*, *[1]*, *[2]*) a los elementos de la lista para calcular el tiempo promedio de respuesta del servidor.
 - d)* Encuentre el tiempo más rápido (mínimo) y el tiempo más lento (máximo) utilizando las funciones propias de Python.
 - e)* Calcule la “brecha de rendimiento”, que corresponde a la resta entre el tiempo máximo y el mínimo.
 - f)* Imprima en pantalla la lista completa de datos y el reporte con el promedio y la brecha calculada.